Command: `sudo yum install git`

Explanation: This command installs Git on a Linux system using the YUM package manager.

- `sudo` – Runs the command with administrator privileges.
- `yum` – Manages software packages.
- `install git` – Downloads and installs Git and its required dependencies.

How it works: The system searches the configured software repositories, downloads Git and any required dependencies, and installs them on the system.

```
[ec2-user@ip-172-31-45-114 ~]$ sudo yum install git
Last metadata expiration check: 3:56:40 ago on Thu Oct 22 05:35:54 2026.
Package git-2.50.1-1.amzn2023.0.1.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-172-31-45-114 ~]$
```

1. Command: `git config --global user.name "username"`

Explanation: Sets your username in Git globally, so Git can identify you as the author of your commits across your repositories.

2. Command: `git config --global user.email "email"`

Explanation: Sets your email address globally in Git. This email is recorded in your commits to identify their author.

3. Command: `git config --list`

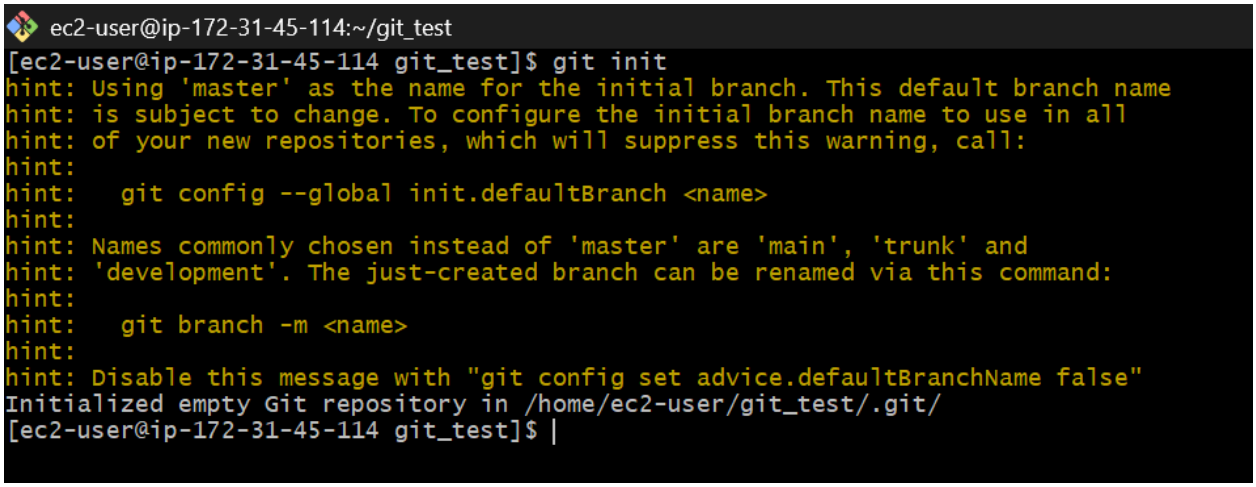
Explanation: Displays the Git configuration settings currently in effect, including your username and email address.

```
ec2-user@ip-172-31-45-114:~/git_test
[ec2-user@ip-172-31-45-114 git_test]$ git config --global user.name "an-techi"
[ec2-user@ip-172-31-45-114 git_test]$ git config --global user.email "anirudh.techi10@gmail.com"
[ec2-user@ip-172-31-45-114 git_test]$ git config --list
user.name=an-techi
user.email=anirudh.techi10@gmail.com
[ec2-user@ip-172-31-45-114 git_test]$
```

Command: `git init`

Explanation: Initializes a new Git repository in the current directory.

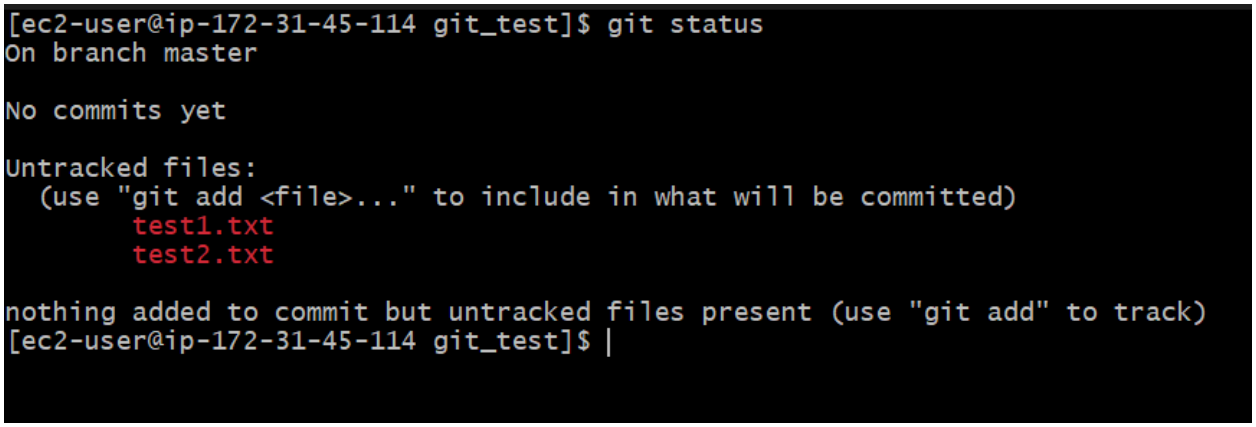
How it works: It creates a hidden `.git` directory that stores the information Git needs to track changes, manage commits, and maintain version history for your project.



Command: `git status`

Explanation: Shows the current status of your Git repository.

How it works: It displays which files are modified, untracked, or staged for commit, helping you understand what changes need to be saved.

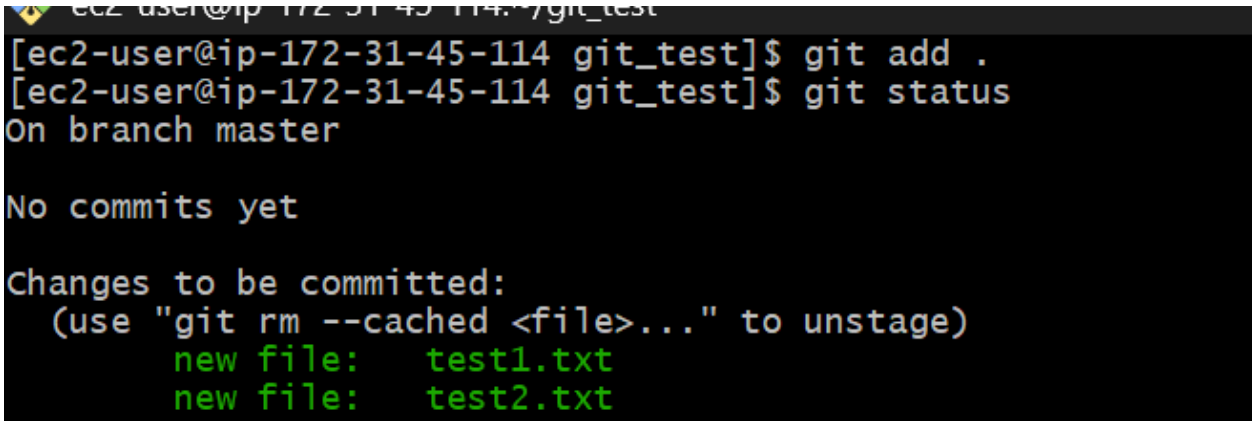


1. Command: `git add filename`

Explanation: Adds a specific file to the staging area, preparing it to be included in the next commit.

2. Command: `git add .`

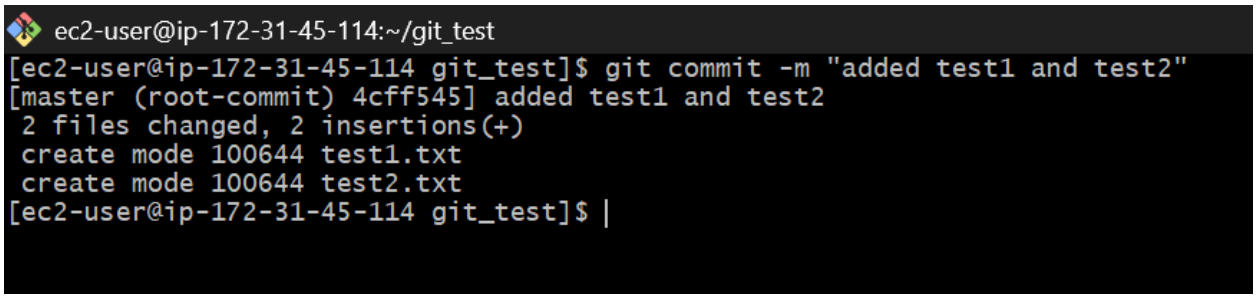
Explanation: Adds all new and modified files in the current directory and its subdirectories to the staging area, preparing them for the next commit.



Command: `git commit -m "message"`

Explanation: Saves the staged changes to the Git repository as a commit.

How it works: The `-m` option lets you add a short message describing the changes, making it easier to track the project's history.



Command: `git log`

Explanation: Displays the commit history of the Git repository.

How it works: It shows details of previous commits, including the commit ID, author, date, and commit message.

```
[ec2-user@ip-172-31-45-114 git_test]$ git log
commit 4cff545394995e6731ce3e8b64dbb1bb3c71c50d (HEAD -> master)
Author: an-techi <anirudh.techi10@gmail.com>
Date:   Thu Oct 22 09:52:05 2026 +0000

    added test1 and test2
[ec2-user@ip-172-31-45-114 git_test]$
```

Command: `ssh-keygen`

Explanation: Generates an SSH key pair consisting of a public key and a private key. These keys are used to securely connect your computer to GitHub.

How it works: The command creates the keys and saves them at the specified location on your computer. You add the public key to GitHub to enable SSH authentication.

Steps to Add the SSH Key to GitHub:

1. Run `ssh-keygen` in the terminal.
2. Press Enter to accept the default file location and follow the prompts.
3. Display the public key using `cat ~/.ssh/id_rsa.pub` or `cat ~/.ssh/id_ed25519.pub`, depending on the key you generated.
4. Copy the entire public key displayed in the terminal.
5. Go to GitHub.com and open **Settings**.
6. Select **SSH and GPG keys** and click **New SSH key**.
7. Enter a title for the key.
8. Paste the copied public key into the Key field.
9. Click **Add SSH key** to save it.

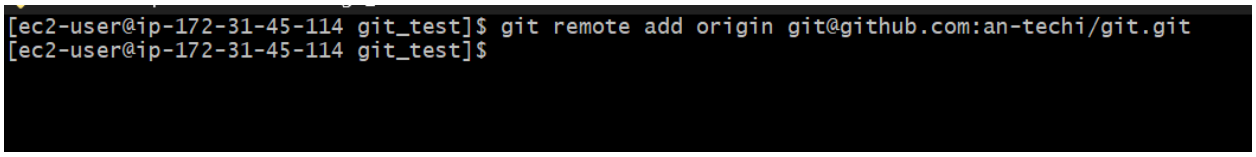
Note: Copy only the public key (`.pub` file). Never share your private key.

```
[ec2-user@ip-172-31-45-114 git_test]$ ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/home/ec2-user/.ssh/id_rsa):
Enter passphrase for "/home/ec2-user/.ssh/id_rsa" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/ec2-user/.ssh/id_rsa
Your public key has been saved in /home/ec2-user/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:s/bf9jV8qbX2z6CaQO+l77u+OK7+qUBn1IxL8lFj/vU ec2-user@ip-172-31-45-114.ap-southeast-2.compute.internal
The key's randomart image is:
```

Command: `git remote add origin sshurl`

Explanation: Connects your local Git repository to a remote repository on GitHub.

How it works: `git remote add` adds a remote repository, `origin` is the name given to that remote, and `sshurl` is the SSH URL of your GitHub repository. This allows you to push and pull code between your local repository and GitHub.



Command: `git remote -v`

Explanation: Displays the remote repositories connected to your local Git repository.

How it works: The `-v` option stands for verbose. It shows the remote name (such as `origin`) and its URLs for fetching and pushing code to and from GitHub.

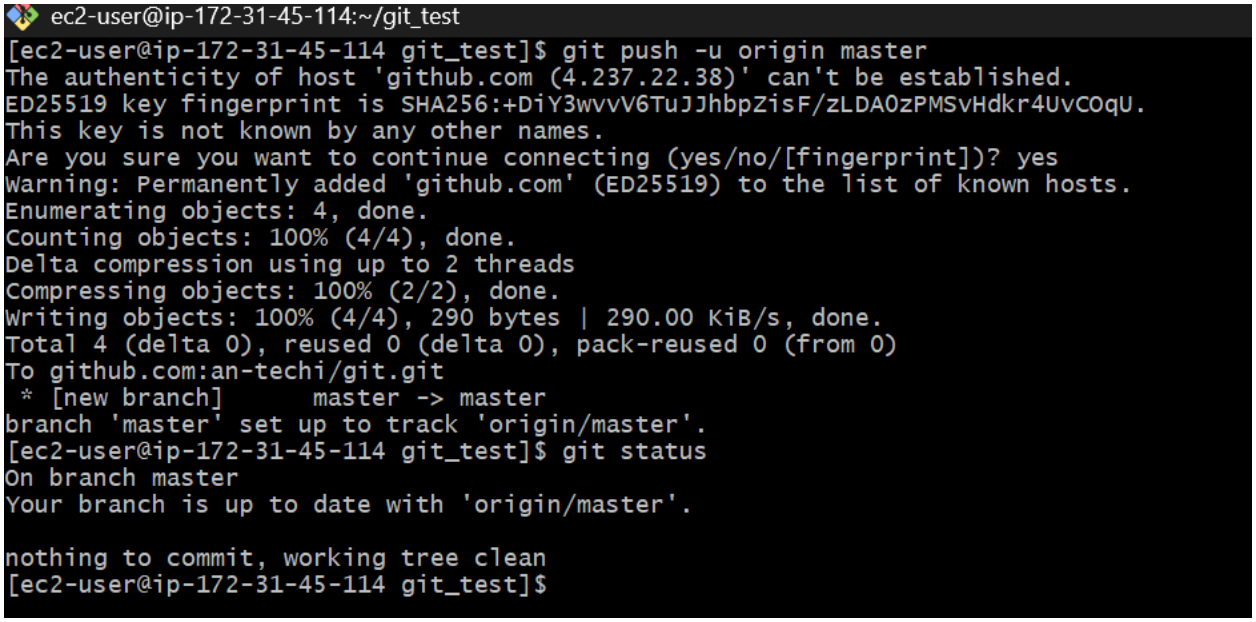


Command: `git push -u origin master`

Explanation: Uploads your local commits from the `master` branch to the remote repository on GitHub.

How it works: `push` uploads commits, `origin` specifies the remote repository, `master` specifies the branch, and `-u` sets the upstream branch so future pushes can usually be done using `git push`.

Note: If your branch is named `main` instead of `master`, use `git push -u origin main`.



Problem: Git Push Rejected (Fetch First)

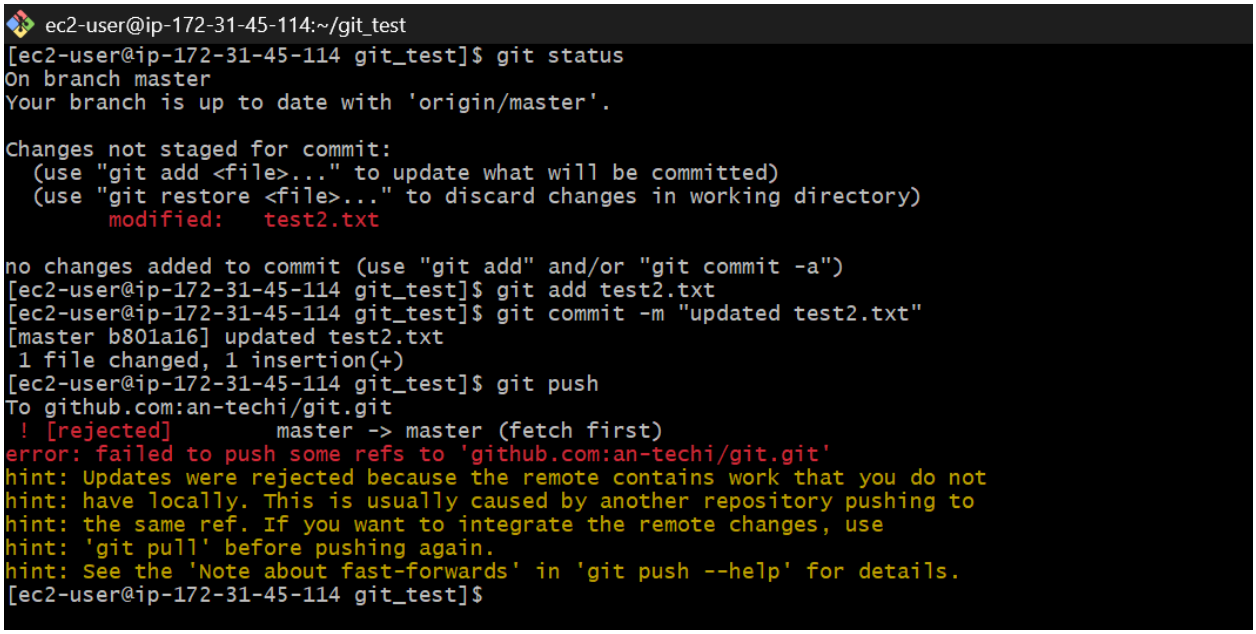
Explanation: This error occurs when the remote repository on GitHub contains changes that are not available in your local repository. Git rejects the push to prevent those changes from being overwritten.

How to Fix It:

1. Pull the latest changes from GitHub:
   `git pull origin master`
2. If Git reports merge conflicts, resolve them and commit the changes if required.
3. Push your changes to GitHub:
   `git push origin master`

How it works: The `git pull` command downloads and integrates the remote changes into your local branch. After integrating those changes, `git push` uploads your local commits to GitHub.

Note: If Git reports unrelated histories or merge conflicts, additional steps may be needed. Avoid force-pushing unless you understand the consequences, because it can overwrite remote history.



Problem: Divergent Branches

Explanation: This error occurs when your local branch and the remote branch on GitHub have different commits. Git needs to know how you want to combine these changes before it can complete the pull.

How to Fix It:

Run the following command to pull and merge the remote changes into your local branch:

```
git pull --no-rebase
```

Then push your changes to GitHub:

```
git push
```

How it works: The `--no-rebase` option tells Git to use merging to combine the local and remote histories. After the merge succeeds, you can push your changes to GitHub.

Note: If merge conflicts occur, resolve them before pushing. You can also use `git config --global pull.rebase false` to make merge the default pull strategy for all your repositories.

```
ec2-user@ip-172-31-45-114:~/git_test
[ec2-user@ip-172-31-45-114 git_test]$ git pull
hint: You have divergent branches and need to specify how to reconcile them.
hint: You can do so by running one of the following commands sometime before
hint: your next pull:
hint:
hint:   git config pull.rebase false  # merge
hint:   git config pull.rebase true   # rebase
hint:   git config pull.ff only       # fast-forward only
hint:
hint: You can replace "git config" with "git config --global" to set a default
hint: preference for all repositories. You can also pass --rebase, --no-rebase,
hint: or --ff-only on the command line to override the configured default per
hint: invocation.
fatal: Need to specify how to reconcile divergent branches.
[ec2-user@ip-172-31-45-114 git_test]$
```

Command: `git pull --no-rebase`

Explanation: Downloads the latest changes from the remote repository and merges them into your current local branch.

How it works: The `--no-rebase` option tells Git to use the merge strategy instead of rebase when integrating remote changes. This helps synchronize your local repository with GitHub while preserving the existing commit history.

```
[ec2-user@ip-172-31-45-114 git_test]$ git pull --no-rebase
Merge made by the 'ort' strategy.
 test_file_in_github | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 test_file_in_github
[ec2-user@ip-172-31-45-114 git_test]$git push
Enumerating objects: 9, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 2 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 544 bytes | 544.00 KiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To github.com:an-techi/git.git
   875f6f9..18e7e7f  master -> master
[ec2-user@ip-172-31-45-114 git_test]$
```

Command: `git log -2`

Explanation: Displays the two most recent commits in the Git repository.

How it works: The `-2` option limits the output to the latest two commits, showing details such as the commit ID, author, date, and commit message.

```
[ec2-user@ip-172-31-45-114 git_test]$ git log -2
commit 18e7e7ffb9b994abb9616540733c4b1bfc1f0135 (HEAD -> master, origin/master, origin/HEAD)
Merge: b801a16 875f6f9
Author: an-techi <anirudh.techi10@gmail.com>
Date:   Thu Oct 22 10:35:31 2026 +0000

    Merge branch 'master' of github.com:an-techi/git

commit b801a167701f4cfd1b070326b5f04a0203433515
Author: an-techi <anirudh.techi10@gmail.com>
Date:   Thu Oct 22 10:25:17 2026 +0000

    updated test2.txt
```

Command: `git revert commitid`

Explanation: Creates a new commit that reverses the changes introduced by the specified commit.

How it works: Replace `commitid` with the actual commit ID you want to undo. Git creates a new commit that reverses those changes without deleting the original commit from the history.

```
ec2-user@ip-172-31-45-114:~/git_test
[ec2-user@ip-172-31-45-114 git_test]$ git revert b801a167701
[master d902561] Revert "updated test2.txt"
 1 file changed, 1 deletion(-)
[ec2-user@ip-172-31-45-114 git_test]$ git status
On branch master
Your branch is ahead of 'origin/master' by 1 commit.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
[ec2-user@ip-172-31-45-114 git_test]$
```

Error: Reverting a Merge Commit

Explanation: This error occurs because the specified commit is a merge commit, which has two parent commits. Git needs to know which parent to keep when reverting the merge.

How to Fix It:

Run the following command:

`git revert -m 1 18e7e7ffb`

How it works: The `-m 1` option tells Git to treat the first parent as the mainline and reverse the changes introduced by the merge relative to that parent.

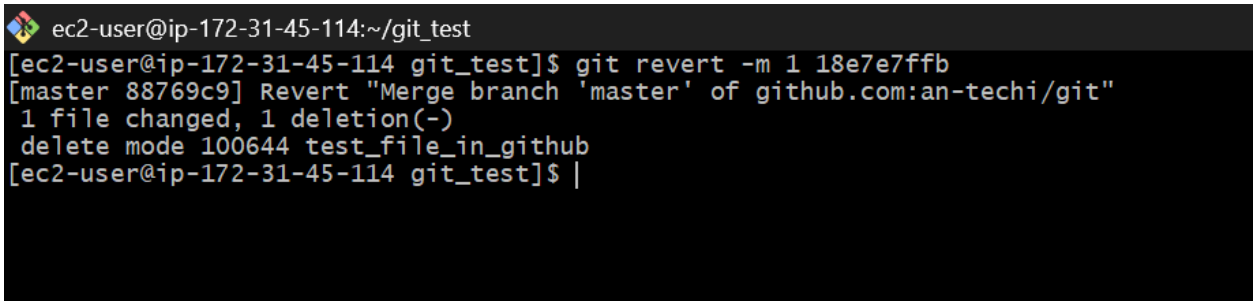
Note: Use `-m 1` only if the first parent is the branch you want to keep as the mainline. Choosing the wrong parent can reverse unintended changes.

```
[ec2-user@ip-172-31-45-114 git_test]$ git revert 18e7e7ffb
error: commit 18e7e7ffb9b994abb9616540733c4b1bfc1f0135 is a merge but no -m option was given.
fatal: revert failed
[ec2-user@ip-172-31-45-114 git_test]$
```

Command: `git revert -m 1 18e7e7ffb`

Explanation: Reverts a merge commit by creating a new commit that undoes the changes introduced by the merge.

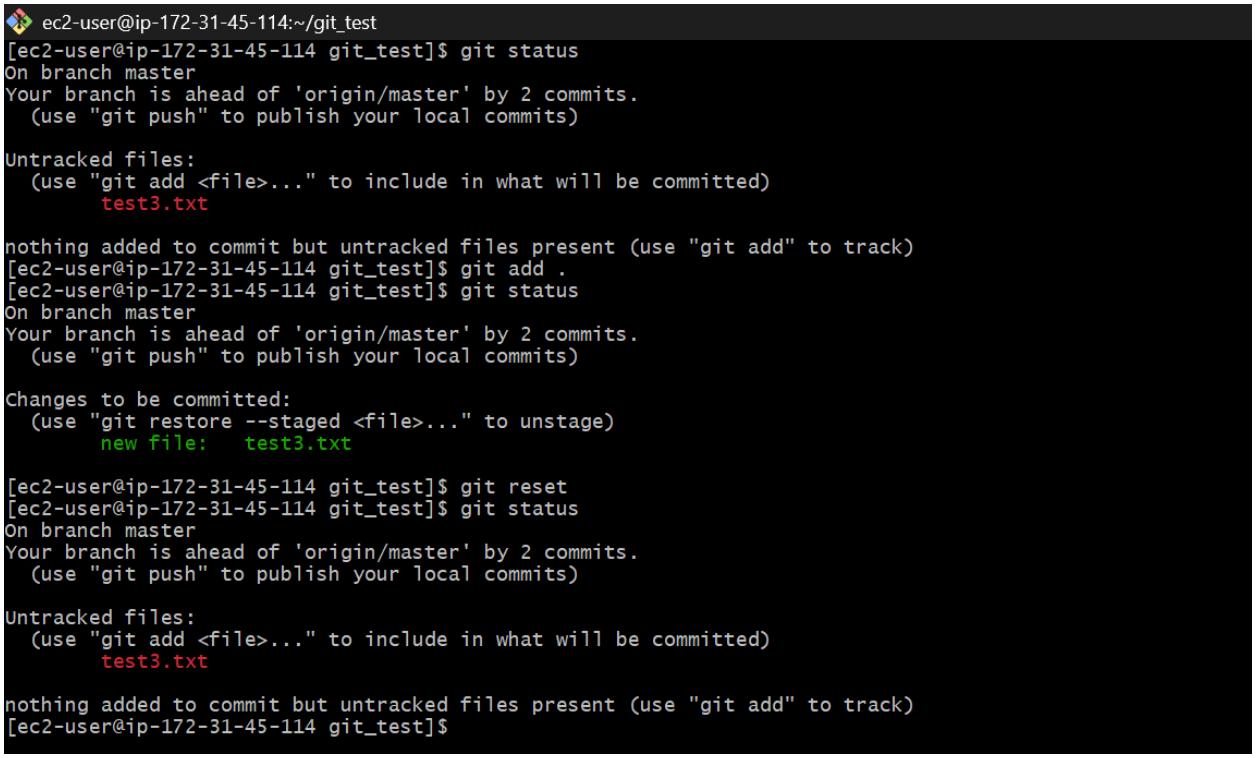
How it works: The `-m 1` option tells Git to use the first parent as the mainline, and `18e7e7ffb` is the commit ID of the merge commit to revert. This preserves the existing history instead of deleting the merge commit.



Command: `git reset`

Explanation: Moves changes out of the staging area, allowing you to unstage files without deleting the changes made to them.

How it works: By default, `git reset` unstages staged changes while keeping your file modifications in the working directory.



Command: `git clone repourl`

Explanation: Downloads a copy of an existing Git repository from a remote server, such as GitHub, to your local computer.

How it works: Replace `repourl` with the repository URL. Git downloads the project files and commit history and creates a local repository that you can work with.

